Offline models and the online test: math and procedure

Status key. built = code exists and was simulated online; **not built** = described as a design only. Numbers quoted are from the paper draft. Items marked *(reasoning, not tested)* are my deduction from the equations.

1 Notation

A *task* is one subject s answering one benchmark b. Items i belong to one benchmark, $b(i)$. A response is $y_{si} \in \{0, 1\}$. Every item has a frozen 1024-d BGE-M3 embedding x_i of its text. At a budget k the *context* is the first k revealed labels $C_k = \{(x_j, y_j)\}_{j \le k}$ and a predictor outputs $\hat{p}(x) = p(y = 1 \mid x, C_k)$ for every unrevealed evaluation item x. Splits are by benchmark: 11 training, 3 validation, 3 test benchmarks. Scores are Brier ALC with budgets $k \in \{0, 1, 3, 7, 15, 31\}$ and weights $w = (0.1, 0.2, 0.2, 0.2, 0.2, 0.1)$.

2 Offline models (fitted on the 11 training benchmarks)

2.1 Offline Rasch with fitted priors [not built]

Model. For training subject s on item i in benchmark $b = b(i)$:

$$\operatorname{logit} P(y_{si} = 1) = \mu_b + \theta_s - b_i. \tag{1}$$

Estimation. Maximise the penalised log-likelihood over all training responses jointly,

$$\max_{\mu,\theta,b} \sum_{(s,i)} \Big[y_{si} \log \sigma(z_{si}) + (1 - y_{si}) \log \big(1 - \sigma(z_{si})\big) \Big] - \sum_b \frac{(\mu_b - m_0)^2}{2\tau_b^2} - \sum_s \frac{\theta_s^2}{2\tau_s^2} - \sum_i \frac{b_i^2}{2\tau_i^2}, \tag{2}$$

with $z_{si} = \mu_b + \theta_s - b_i$. The hyperparameters $(m_0, \tau_b, \tau_s, \tau_i)$ would be set by empirical Bayes: m_0 is the mean of the fitted $\hat{\mu}_b$ and each τ the spread of its fitted parameters (maximising the marginal likelihood is the better estimator, since a raw spread includes estimation noise).

Optional feature link (explanatory IRT): $b_i = w^\top x_i + u_i$, $u_i \sim \mathcal{N}(0, \tau_i^2)$, with x_i centred within its benchmark and w fitted by ridge on the training benchmarks. Our probe of this idea transfers poorly (OOD rank correlation 0.09).

What survives to a test benchmark. Only the hyperparameters, and w if used. The fitted $\hat{\mu}_b, \hat{\theta}_s, \hat{b}_i$ belong to training benchmarks, subjects and items; none of them occurs in test.

What offline fitting can do that the online fit cannot *(reasoning, not tested)*: in training, a benchmark has many subjects, so μ_b (benchmark average) and θ_s (deviation from it) are separately identified, and τ_b, τ_s can be estimated separately. In a competition-shaped draw there is one subject per benchmark, so the data fix only $\mu_b + \theta_s$ and the prior alone splits it.

2.2 Hierarchical Beta-Binomial with subject metadata [built]

Task accuracy $\pi \sim \text{Beta}(\kappa\mu_s, \kappa(1 - \mu_s))$, so with h hits in k labels

$$\hat{p} = \frac{\kappa\mu_s + h}{\kappa + k}, \qquad \mu_s = \sigma\big(\beta_0 + e_{\text{name}} + e_{\text{provider}} + w \cdot \text{date} + v \cdot \text{date_missing}\big). \tag{3}$$

Name and provider embeddings, w, v and the single κ are fitted by Beta-Binomial marginal likelihood on the training tasks; unseen names map to a learned "unknown"; the L_2 strength is chosen on validation. At $k = 0$ it predicts μ_s. On test it did not beat the plain Beta-Binomial reliably (the release-date term extrapolates).

2.3 Linear probe for item difficulty [built; a diagnostic, not a predictor]

Ridge regression $\hat{d}_i = \beta^\top x_i$ from the item embedding to the benchmark-centred observed pass rate, penalty chosen by cross-validation. ID: fit on 80% of a training benchmark's items, score the rest. OOD: fit on training benchmarks, score each unseen benchmark. Scored by within-benchmark Spearman rank correlation and pairwise accuracy.

3 Online testing procedure

3.1 Protocol for one task

1. Split the task's items once, 50/50, into an *acquisition pool* and an *evaluation pool*. The evaluation labels are never revealed.
2. Fix a candidate order over the acquisition pool (3 frozen orders per validation task, 10 per test task), so every model sees the same labels in the same order.
3. Reveal labels one at a time. After $k = 0, 1, 3, 7, 15, 31$ labels, the predictor outputs $\hat{p}$ for every evaluation item. Score the Brier $\frac{1}{|E|}\sum_{i\in E}(\hat{p}_i - y_i)^2$.
4. Combine budgets: $\text{ALC} = \sum_k w_k \,\text{Brier}_k$.

3.2 Single-pair, multi-pair and competition-shaped draws

- **Single-pair:** each task runs alone; the predictor sees only its own labels (82 validation, 45 test tasks).
- **Multi-pair:** a *draw* is a random group of tasks with one or two subjects per benchmark (3–6 tasks); the predictor sees every task's revealed labels (60 validation, 150 test draws).
- **Competition-shaped:** multi-pair with exactly one subject per benchmark (120 validation, 150 test draws).

Example test draw: `algotune` (subject A), `cmmlu` (subject B), `editbench` (subject C): at budget k the predictor holds up to $3k$ labels in total.

3.3 How each model is run online

Beta-Binomial (exact Bayesian sequential update). State (h, n) per task; each label adds 1 to n and y to h. Posterior $\text{Beta}(\frac{1}{2} + h, \frac{1}{2} + n - h)$; prediction $\hat{p} = (h + 0.5)/(n + 1)$, the same for every item of the task. No offline data.

Pooled Rasch (online MAP, no offline training). At each budget, with the labels $\mathcal{D}$ revealed so far in the *draw*, solve

$$\max_{\mu,\theta,b} \sum_{(b,s,i,y)\in\mathcal{D}} \log p(y \mid z) - \sum_b \tfrac{(\mu_b - m_0)^2}{2\tau_b^2} - \sum_s \tfrac{\theta_s^2}{2\tau_s^2} - \sum_i \tfrac{b_i^2}{2\tau_i^2}, \quad z = \mu_b + \theta_s - b_i, \tag{4}$$

by Newton's method ($\theta^{t+1} = \theta^t - H^{-1}g$ with g the penalised gradient and H its Hessian), starting from the prior means, discarded after predicting. Predict $\hat{p} = \sigma(\hat{\mu}_b + \hat{\theta}_s - \hat{b}_i)$, with $\hat{b}_i = 0$ for an item no one has labelled. Fixed: $m_0 = 0.049$, $\tau_b = 2$, $\tau_s = 1$, $\tau_i = 1$. Items are matched across subjects by content. Ablating μ_b or θ_s means setting that width near zero.

Kernel Rasch. Same, but within each benchmark the item prior is joint, $\mathbf{b}_b \sim \mathcal{N}(0, K_b)$ with $K_{ij} = \tau^2[(1-\rho)\delta_{ij} + \rho\, e^{(\cos(x_i, x_j)-1)/T}]$: the item penalty becomes $\frac{1}{2}\mathbf{b}_b^\top K_b^{-1}\mathbf{b}_b$. An unlabelled item gets $\hat{b}_i = K_{iL}K_{LL}^{-1}\hat{\mathbf{b}}_L$ (kriging) from the labelled items L. Fixed: $\tau = 1.4$, $\rho = 0.8$, $T = 0.2$.

Offline Rasch with fitted priors [not built]**.** The pooled Rasch online fit above, with $(m_0, \tau_b, \tau_s, \tau_i)$ (and, if used, a prior mean $w^\top x_i$ for b_i) taken from the offline fit instead of picked on validation.

Algorithm 1 Online test of one predictor on one draw

Require: draw $\mathcal{G}$ of tasks; per task: acquisition pool, evaluation pool, frozen order; budgets $\{0, 1, 3, 7, 15, 31\}$

for each budget k **do**
  $\mathcal{D}_k \leftarrow$ first k labels of every task's order ▷ labels revealed so far
  predictor produces $\hat{p}_i$ for every evaluation item of every task in $\mathcal{G}$, using $\mathcal{D}_k$ (own-task labels only in single-pair runs)
  for each task $t \in \mathcal{G}$ **do**
    $\text{Brier}_{t,k} \leftarrow$ mean squared error on t's evaluation items
  end for
end for
$\text{ALC}_t \leftarrow \sum_k w_k \,\text{Brier}_{t,k}$ for each task; average over tasks and draws

3.4 Statistics

For each model the per-task ALC is stored. Differences between two models are computed per task (paired) and resampled with replacement over tasks (thousands of resamples); the 95% CI is the 2.5th–97.5th percentile. A difference is called significant when the CI excludes zero. The resampling unit is the task, not the benchmark, with three test benchmarks, so these intervals understate the uncertainty about new benchmarks.

4 What each model carries from offline to online

Model	Offline fit	Carried into the online test	Status
Beta-Binomial	none	prior Beta($\frac{1}{2}, \frac{1}{2}$) only	built
Pooled / kernel Rasch	none	m_0 (training mean), prior widths chosen on validation; kernel: τ, ρ, T	built
Offline Rasch, fitted priors	joint MAP over training tasks	$m_0, \tau_b, \tau_s, \tau_i$ (and w if used); no fitted μ_b, θ_s, b_i	not built
Hierarchical Beta-Binomial	marginal likelihood on training tasks	κ, metadata coefficients, embeddings	built